

```

// Carmen -v33
// Phone-first AI research workspace. Paste into Cloudflare Workers > Edit code.

const ASSETS = {
  "/": "<!doctype html><html lang=\"en\"><head><meta charset=\"utf-8\"><meta name
  \"/index.html\": "<!doctype html><html lang=\"en\"><head><meta charset=\"utf-8\">
  "/app.js": "const S={tab:'home',results:[],saved:[],messages:[],investigation:n
  \"/manifest.webmanifest": "{ \"name\": \"Carmen\", \"short_name\": \"Carmen\", \"star
  \"/icon.svg\": "<svg xmlns=\"http://www.w3.org/2000/svg\" viewBox=\"0 0 180 180\"
  \"/sw.js": "const C='carmen-v33';self.addEventListener('install',e=>e.waitUntil(
  };

const TYPES = {
  "/": "text/html; charset=utf-8",
  "/index.html": "text/html; charset=utf-8",
  "/app.js": "application/javascript; charset=utf-8",
  "/manifest.webmanifest": "application/manifest+json; charset=utf-8",
  "/icon.svg": "image/svg+xml",
  \"/sw.js": "application/javascript; charset=utf-8"
};

async function searchWeb(req){const u=new URL(req.url);const q=u.searchParams.get
async function chat(req,env){try{if(!env.API_KEY)throw Error('Worker API_KEY secr

export default {
  async fetch(req, env) {
    const u = new URL(req.url);
    if (req.method === 'OPTIONS') return new Response('', { headers: cors(req) })
    if (u.pathname === '/health') return json({ ok:true, provider:'openai-compati
    if (u.pathname === '/search' && req.method === 'GET') return searchWeb(req);
    if (u.pathname === '/chat' && req.method === 'POST') return chat(req,env);
    if (u.pathname === '/analyze' && req.method === 'POST') return analyze(req,en
    if (u.pathname === '/synthesize' && req.method === 'POST') return synthesize(
    let path=u.pathname;
    if(path.endsWith('/')) path='/';
    const body=ASSETS[path] ?? ASSETS['/'];
    return new Response(body,{headers: {'content-type':TYPES[path]}| 'text/plain; c
  }
};

async function analyze(req,env){
  try{
    const body=await req.json();

```

```

    validateImage(body.imageUrl);
    if(!env.API_KEY) throw Error('Worker API_KEY secret is not configured');
    const prompt=`You are Carmen, a conservative visual-evidence analyst. Return
    const parsed=await provider(env,[{type:'text',text:prompt},{type:'image_url',
    return json(parsed,200,req);
  }catch(e){return json({error:e.message},500,req)}
}

```

```

async function synthesize(req,env){
  try{
    const body=await req.json();
    if(!Array.isArray(body.references)||body.references.length<2) throw Error('Se
    if(body.references.length>4) throw Error('Compare up to 4 references at once.
    if(!env.API_KEY) throw Error('Worker API_KEY secret is not configured');
    const content=[{type:'text',text:`You are Carmen performing conservative evid
    body.references.forEach((r,i)=>{
      content.push({type:'text',text:`REFERENCE ${i+1}: ${String(r.title||'Un titl
      if(typeof r.imageUrl==='string' && r.imageUrl.startsWith('data:imag
    });
    const parsed=await provider(env,content);
    return json(parsed,200,req);
  }catch(e){return json({error:e.message},500,req)}
}

```

```

function validateImage(x){
  if(typeof x!=='string'&&!x.startsWith('data:image/')) throw Error('imageUrl
  if(x.length>16_000_000) throw Error('Image is too large. Use a smaller screensh
}

```

```

async function provider(env,content){
  const r=await fetch(env.API_URL||'https://api.openai.com/v1/chat/completions',{
  const text=await r.text();let j;try{j=JSON.parse(text)}catch{throw Error(text||
  if(!r.ok) throw Error(j?.error?.message||JSON.stringify(j));
  const out=j.choices?.[0]?.message?.content||'';const clean=out.replace(/`^```jso
  try{return JSON.parse(clean)}catch{throw Error('Vision provider returned invali
}

```

```

function cors(req){const origin=req.headers.get('Origin');return {'access-control
function json(x,s,req){return new Response(JSON.stringify(x),{status:s,headers:{.

```